1. Понятие за софтуерна архитектура. Структури и изгледи (structures and views) на архитектурата. Необходимост от софтуерни архитектури. Влияние на архитектурата върху проекта и организацията.

Дефиниция и същност

Софтуерната архитектура на една система представлява съвкупност от структури, които показват различните софтуерни елементи, техните външно видими свойства и връзките помежду им. Тя е **абстракция**, която разглежда елементите като „черни кутии“, фокусирайки се върху техните взаимодействия, а не върху вътрешната им реализация (алгоритми и детайли).

Архитектура на дадена софтуерна система е съвкупност от изгледи, показващи различните софтуерни елементи на системата, външното видимите им свойства и връзките между тях.

Структури и изгледи (Structures and Views)

Важно е да се прави разлика между структура и изглед:

- **Структура:** Реалната съвкупност от софтуерни елементи и техните връзки.
- **Изглед (View):** Конкретно документирано представяне на дадена структура, предназначено за комуникация със заинтересованите лица.

Структура ⇔ изглед, приема се като едно и също нещо и ползваме надолу едното, не двете, но има разлика и трябва да подчертаем

Основни групи архитектурни структури:

1. **Модулни структури:** Описват функционалните компоненти (например модули за продукти, плащания, потребители).
    a. Декомпозиция на модулите
    b. Употреба на модулите
    c. Структура на слоевете (многослоест)
    d. Йерархия на класовете
2. **Структури на процесите:** Описват динамичното поведение и комуникацията по време на изпълнение (например как заявка преминава през слоевете).
    a. Състояния при които преминава нашата система по време на изпълнение.
3. **Структури на разположението (Физически):** Показват разпределението на софтуерните компоненти върху хардуерни ресурси (сървъри, бази данни, кеш сървъри).
    a. Структура на внедряване
    b. Файлова структура
    c. Разпределение на работата

***Модел 4+1*:**

- **Логически изглед:** Абстракции като обекти и класове.
- **Изглед на процесите:** Взаимодействащи си процеси в реално време.
- **Изглед на кода:** Организация на файловете и пакетите.
- **Физически изглед:** Разположение върху хардуерни възли.
+ *Сценарий на реализация*

Необходимост и влияние

Архитектурата е необходима за управление на сложността в големи системи. Тя осигурява обща картина и език за комуникация между разработчици, тестери и мениджъри.

Architecture business cycle

Бизнесът влияе върху архитектурата чрез:

- изисквания;
- бюджет;
- срокове.

Архитектурата влияе върху:

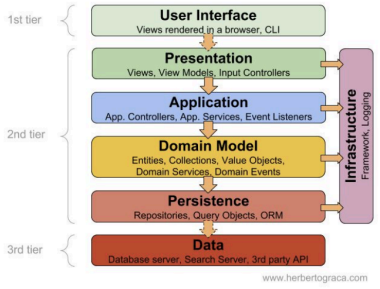
- организацията;
- риска;
- разходите;
- поддръжката.

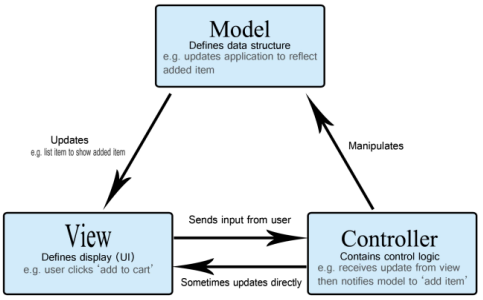
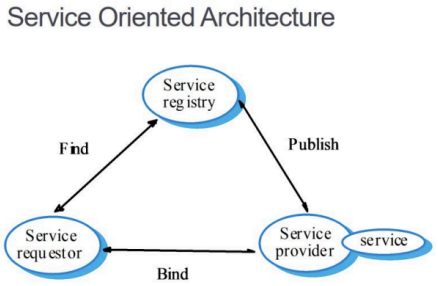
2. Архитектурни стилове. Многослоен стил, Модел-изглед контролер (MVC), Поточни (Pipe-and-filter) архитектури. Архитектура ориентирана към услуги.

Архитектурните стилове са общи шаблони за организация, които влияят пряко върху качествените характеристики на системата.

Стил е набор от:

- *Компоненти*
- *Конектори*
- *Ограничения*
- *Правила за взаимодействие*

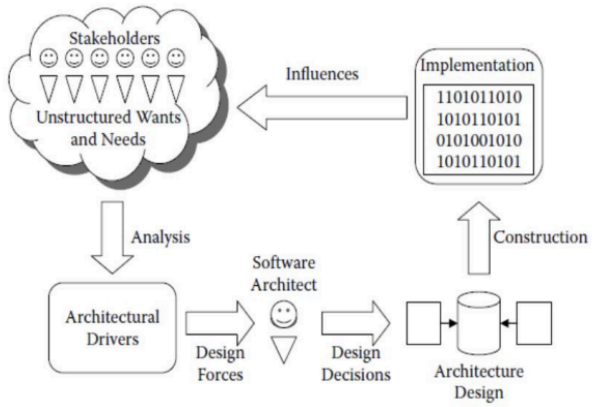
Стил	Описание	Предимства/Недостатъци
Многослоен (Layered) ***Слой - използва услуги само на слоя под него, предоставя услуги на слоя под него***	Вертикално разделени слоеве. Всеки слой предлага услуги на горния и ползва долния.	• Ясно разделение на отговорностите; • риск от намалена производителност • *скриване на информацията;* • *висока кохезия;* • *лесна поддръжка;* • *допълнителни зависимости като недостатък.* *Когато искаме да разделяме отговорности, за по-лесно тестване тъй като може поотделно да тестваме* 

MVC (Model-View-Controller)	Разделя данните (Модел), представянето (Изглед) и логиката на управление (Контролер).	Позволява една логика да има различни интерфейси; улеснява тестването. *Допълнителна сложност, допълнителен код* 
Поточен (Pipe-and-filter)	Последователност от филтри, свързани чрез канали (pipes). Всеки филтър трансформира данни - *получава ги, обработва ги и подава резултата към следващия компонент*	Възможност за паралелизъм и повторна употреба; разходи за парсване на данни. *Трудно споделя…*
Ориентиран към услуги (SOA)	Система от самостоятелни услуги със строго дефинирани интерфейси (договори). *Service Provider, Service Registry, Service Consumer. Стъпва върху Клиент-сървър, ефективно наследява неговите характеристики* *основни операции:* ● ***Publish**: регистриране на услуга;* ● ***Find**: откриване на услуга;* ● ***Bind**: използване на услуга.* *характеристики:* ● *повторна употреба;* ● *оперативна съвместимост*	Гъвкавост и слаба свързаност; сложност при мрежова комуникация. 

3. Проектиране на софтуерната архитектура. Процес за проектиране. Избор на подходящи структури. Последователност на създаване на архитектурата. Тактики (архитектурни решения) за постигане на желаните качествени показатели.

Процес на проектиране

Проектирането е **цикличен процес**, който започва с анализ на нуждите на заинтересованите лица (stakeholders).



1. **Идентифициране на драйвери:** Избор на най-приоритетните функционални изисквания (use-cases) и сценарии за качество.
2. **Декомпозиция:** Разделяне на системата йерархично на модули.
3. **Избор на тактики и шаблони:** Прилагане на конкретни решения за постигане на драйверите.
4. **Валидация:** Проверка дали декомпозицията отговаря на изискванията.

Входни данни:

- *функ. изиск., качествени*

Драйвери

конкретни примери:

- *функционални изисквания: какво трябва да прави системата;*
- *качествени показатели: производителност, сигурност, надеждност;*
- *бизнес ограничения: срокове, бюджет, технологии.*

ADD (attribute driven design)

ADD е метод за проектиране на архитектурата въз основа на качествените показатели.

Стъпки:

1. *избор на елемент за декомпозиция;*
2. *избор на архитектурни драйвери;*

3. *избор на архитектурен стил;*
4. *създаване на подмодули;*
5. *дефиниране на интерфейси;*
6. *проверка на решенията.*

Тактики за качествени показатели

Същност на арх. Тактики - Архитектурно решение за постигане на конкретен качествен показател
Набор от тактики наричаме арх. Стратегя. Ползваме това за да избегнем сценарий за качество

Тактиките са конкретни технически решения:

- **Изправност (Reliability):** Ping/echo - изпращане сигнал, Heartbeat - изпращане периодичен сигнал; активен/пасивен излишък (redundancy)*, exceptions; извеждане на отказал компонент от употреба; process monitoring; shadow mode; checkpoint/rollback.*
- **Производителност:** Кеширане, Load-balancing (разпределение на натоварването), паралелна обработка, *арбитраж (scheduling: FIFO), по-добри алгоритми; намаляване на overhead; ограничаване на времето; реплики; добавяне на повече ресурси; приоритетно планиране.*
- **Сигурност:** Криптиране, хеширане на пароли, двуфакторна автентикация (2FA), *authentication, authorization, integrity; възстановяване след атака (audit trail) - възстановяване на състояние и ….*
- **Скалируемост:** Хоризонтално (scaling out) и вертикално (scaling up) скалиране, sharding на бази данни.
- **Изменяемост/*Modifiability*:** Прилагане на принципа Open-Closed; *Локализиране на промените (поддържане на семантична свързаност; очакване на бъдещи промени; ограничаване на опциите.); Предотвратяване на ripple effect (скриване на информацията; ограничена комуникация; посредници.); Отлагане на свързването (конфигурационни файлове; Plug-and-Play компоненти.)*
- ***Testability*** *(Record and play - следване на стъпки които се изпълняват в системата, Interface vs implementation - отделно тестване на двете, test interface - създаване на интерфейс за тестване, мониторинг - следване на стъпки които се изпълняват в системата)*
- ***Usability*** *(Feedback, Undo/cancel, multiple views, user initiative vs system initiative - leading the user; тактики, свързани с UI и насочени към разработчика на интерфейса.)*

4. Документиране на софтуерната архитектура. Предназначение на документацията. Основен принцип на документиране, съдържание и структура на документацията.

Предназначение и принципи

Документацията служи за комуникация, поддръжка, архитектурен анализ, съхраняване на знания, обучение на нови кадри и основа за разработка.

- **Основен принцип:** Архитектурата се документира чрез множество изглед и документацията трябва да е съобразена с нейните читатели. Тя трябва да е точна, актуална и да обяснява не само „какво“, но и „защо“ е взето дадено решение.

Съдържание и структура на документацията

Структура:

- **основна информация**: цели, контекст, терминология;
- **архитектурни изгледи**: модулни, процесни, разположение;
- **Beyond Views**: решения, ограничения, обосновка.

Един стандартен архитектурен документ съдържа:

1. **Въведение:** Цели и обхват.
2. **Архитектурни изгледи:** Детайлни описания (логически, физически и др.) - *модулни, процесите, разположението.*
    a. *Основно предстявяне*
    b. *Елемент и връзки*
    c. *Контекст*
    d. *Варианти на използване*
    e. *Архитектурна обосновка*
    f. *Терминологичен речник*
    g. *Допълнителна информация*
3. **Архитектурни решения:** Обосновка на избраните стилове и тактики (включително разгледани алтернативи).
4. **Елементи и връзки:** Детайлно описание на интерфейсите.
5. **Терминологичен речник:** Дефиниции за избягване на двусмислие.